

Coprocessador RSA · AMBA 3 APB · TSMC 28 nm

Design Review — RSA_APB

Autores

Matheus Grossi · Milena Resmer

Gerente

Alcides Costa

TchêMOS — Semiconductor Design House

Novembro de 2026

UNISINOS
UNIVERSIDADE DO VALE DO RIO DOS SINOS

GOVERNO DO ESTADO
RIO GRANDE DO SUL
SECRETARIA DE INOVAÇÃO, CIÊNCIA E TECNOLOGIA

Sumário

1. Entregáveis e arquitetura 2
2. E01 — Coprocessador RSA de 32 bits e testbench 8
3. E02 — Equivalência lógica RSA 4 bits 10
4. E03 — Simulação funcional do `rsa_core32b` 12
5. E04 — Ambiente de verificação UVM 14
6. E05 — Simulação funcional do RSA_APB 17
7. E06 — Cobertura funcional do RSA_APB 19
8. E07 — Gate-level netlist TSMC 28 nm 20
9. E08 — Análise estática de temporização 23
10. Comparativo MG × MR — síntese TSMC 28 nm 27
11. E09 — Simulação pós-síntese com SDF 28
12. E10 — Scripts de automação 30
13. Falhas encontradas no RSA32 34

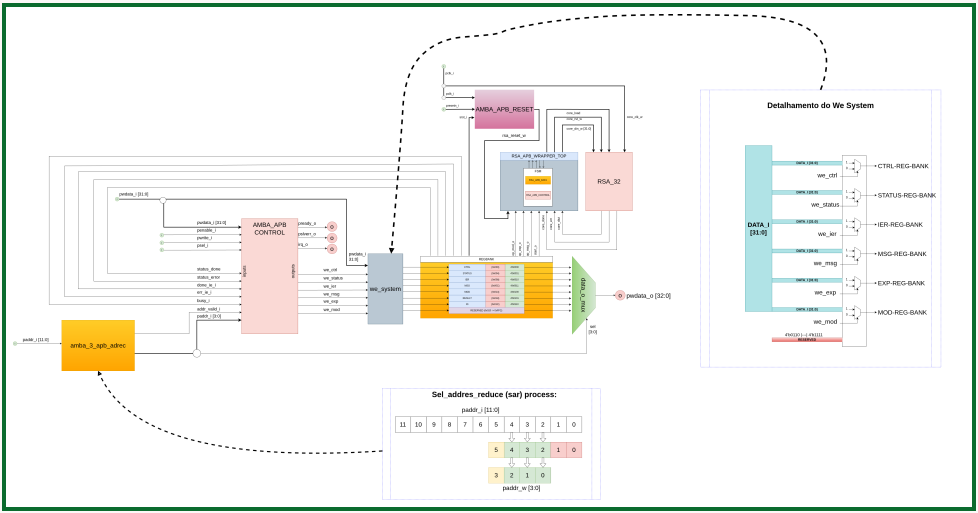
Entregáveis do projeto

Critérios de aceitação

ID	Entregável	Requisito de aceitação
E01	Coprocessador RSA de 32 bits e respectivo testbench em SystemVerilog	Código-fonte disponível e funcional
E02	Relatório de equivalência lógica entre as implementações RSA de 4 bits em Verilog e SystemVerilog	Equivalência lógica comprovada
E03	Relatório de simulação funcional do `rsa_core32b`	Testes executados com sucesso
E04	Ambiente de verificação UVM para a interface `rsa_apb`	Ambiente funcional e executável
E05	Relatório de simulação funcional da interface RSA_APB de 32 bits	Testes funcionais aprovados
E06	Relatório de cobertura funcional da interface RSA_APB de 32 bits	Métricas apresentadas e correlacionadas ao plano de verificação
E07	Gate-level netlist mapeado para a tecnologia TSMC de 28 nm	Netlist gerado corretamente pelo Cadence Genus
E08	Relatório de análise estática de temporização (STA)	Ausência de violações de setup nos caminhos analisados (TNS = WNS = 0)
E09	Relatório de simulação pós-síntese da interface RSA_APB, com anotação de atrasos SDF	Testes aprovados, sem violações de temporização reportadas durante a simulação
E10	Scripts de automação do fluxo de desenvolvimento	Execução reproduzível das etapas e geração dos respectivos artefatos

Rascunho da arquitetura

Concepção inicial do periférico RSA_APB



MG · um módulo por função · 15 módulos

```
amba_3_apb_rsa_apb_top
├── amba_3_apb_data_io_top
│   ├── amba_3_apb_adrec
│   └── amba_3_apb_data_o_mux
├── amba_3_apb_control
├── amba_3_apb_regbank
├── amba_3_apb_reset
└── amba_3_apb_rsa_wrapper_top
    ├── ..._rsa_wrapper_control
    ├── ..._rsa_wrapper_data
    └── rsa_core (v2)
        ├── rsa_core_ctrl + KSA 32b
        ├── rsa_core_mult + KSA 64b
        └── rsa_core_mod  + KSA 64b, 33b
```

MR · escravo APB monolítico · 8 módulos

```
amba_3_apb_rsa_apb_top  (+ srst_cnt, mux)
├── rsa_apb_rst_sync
├── rsa_apb_regs
├── rsa_apb_seq
└── rsa_core  (ctrl, mult, mod)
```

Correspondência entre os blocos

Função	MG	MR
Decodificação de endereço	`adrec`	`rsa_apb_regs` (`word == W_x`)
Mux de PRDATA	`data_o_mux` + gate em `data_io_top`	`always_comb` em `rsa_apb_regs`
Fase APB, PSLVERR, write enables	`control`	`wr_acc`, `wr_ok`, `pslverr` em `rsa_apb_regs`
IRQ	`control`	`rsa_apb_regs`
Registradores e STATUS	`regbank`	`rsa_apb_regs` (um `always_ff`)
Comandos START/SRST	`regbank` (combinacional)	`start_req`, `srst_req`
Reset do núcleo	`reset`	`rst_sync` + `srst_cnt` no top
Sequenciador de cargas	`wrapper_control` (estado) + `wrapper_data` (saídas)	`rsa_apb_seq`
Seleção do operando	`core_din_o` em `wrapper_data`	`operand_sel` + mux no top
Somadores do núcleo	Kogge-Stone explícito (4 inst.)	+/− (escolha do Genus)

Mesmas portas no top e mesmo núcleo algorítmico (FSMs `ctrl`/`mult`/`mod`); a diferença está no particionamento da interface APB e do sequenciador. KSA = `kogge_stone_adder`.

MG · `amba_3_apb_control.sv`

```
// APB Phase Encoding:
localparam logic [3:0] IDLE   = 4'b1000,
                       SETUP  = 4'b1001,
                       ACCESS = 4'b1011;
...
// APB Phase Decoder:
always_comb
    begin : control_branch
        if (psel_i && !penable_i)
            apb_phase = SETUP;
        else if (psel_i && penable_i)
            apb_phase = ACCESS;
        else
            apb_phase = IDLE;
    end
```

Decodificador de fase explícito (`IDLE`/`SETUP`/`ACCESS`). É **combinacional**, não uma FSM registrada: a fase é recalculada a cada ciclo a partir de `psel` e `penable`.

MR · `rsa_apb_top.sv` (`rsa_apb_regs`)

```
// The transfer completes in the access phase
// (PREADY is always high)
wire wr_acc = psel && penable && pwrite;
...
wire wr_ok = wr_acc && !ro_write && !busy_write;

assign pready = 1'b1;
```

Não nomeia fases: qualifica a escrita direto por `psel && penable && pwrite`.

Em comum

`PREADY = 1` fixo nas duas (sem *wait states*); por isso `APB_WAIT_HOLD` não é exercitada na UVM.

Condição	MG	MR
Endereço reservado (leitura ou escrita)	**1**	0
Escrita em RESULT/ID	1	1
Escrita em MSG/EXP/MOD com BUSY	1	1
Escrita de START com BUSY	**0**, START ignorado*	1, sem efeito
Escrita de SRST com BUSY	0, SRST executado (log UVM)	0, SRST executado

* Pela leitura do RTL; nenhuma bancada exercita START com BUSY.

Endereço reservado

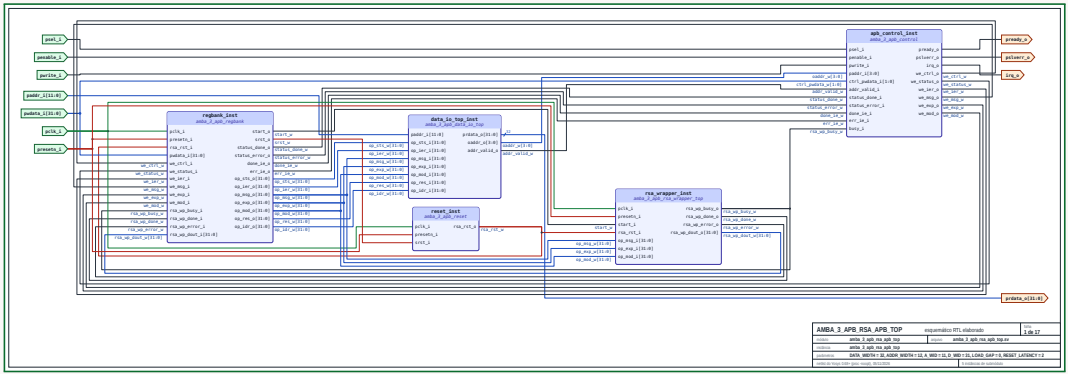
No MG, `addr_valid = ~|paddr[11:5]` (`adrec`) alimenta o PSLVERR em qualquer acesso a 0x020–0xFFF. O MR não tem sinal de validade: endereço que não casa com nenhum registrador é ignorado, como na lista da seção 6.7 da especificação.

CTRL com BUSY no MG

O ramo de `control` testa `ctrl_pwdata == 2'b10 && busy`, ou seja, **SRST**, embora o comentário diga START. Ele nunca dispara: o SRST força `rsa_rst`, que zera BUSY no mesmo ciclo. START com BUSY é só descartado no `regbank`, sem PSLVERR.

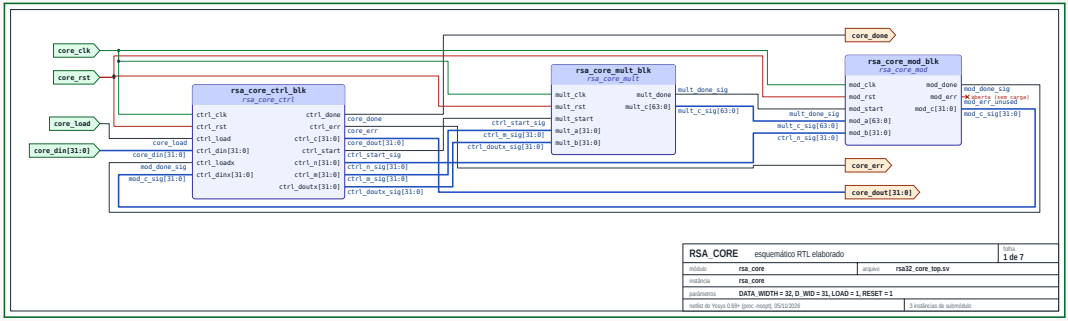
Diagrama de blocos do topo

amba_3_apb_rsa_apb_top — esquemático RTL elaborado



E01 — Coprocessador RSA de 32 bits e testbench

rsa_core — esquemático RTL elaborado do topo



E01 — Coprocessador RSA de 32 bits e testbench

Recorte do RTL · núcleo v2 (RSA32_opt) · SystemVerilog

Topo `rsa_core` · `rsa32_core_top.sv`

```
module rsa_core #(
    parameter   DATA_WIDTH = 32,
                D_WID      = (DATA_WIDTH-1),
                RESET = 1'b1,
                LOAD = 1'b1
)
(
    input   logic core_clk,
    input   logic core_rst,
    input   logic core_load,
    input   logic [D_WID:0] core_din,
    output  logic core_done,
    output  logic core_err,
    output  logic [D_WID:0] core_dout
);
    ...
    rsa_core_mult #( ... ) rsa_core_mult_blk (
        .mult_start (ctrl_start_sig),
        .mult_a     (ctrl_m_sig),
        .mult_b     (ctrl_doutx_sig),
        .mult_done  (mult_done_sig),
        .mult_c     (mult_c_sig), ... );
    rsa_core_mod #( ... ) rsa_core_mod_blk (
        .mod_start  (mult_done_sig),
        .mod_a      (mult_c_sig), ... );

    rsa_core_ctrl #( ... ) rsa_core_ctrl_blk ( ... );
endmodule
```

Controle `rsa_core_ctrl` · `rsa32_core_ctrl.sv`

```
// CASE2 subtracts two; CASE1 and START subtract one.
assign e_decrement_amount = (state_reg == CASE2) ? DATA_WIDTH'(2) : DATA_WIDTH'(1);
kogge_stone_adder #(
    .DATA_WIDTH(DATA_WIDTH)
) u_exponent_kogge_stone (
    .a_i    (e_reg),
    .b_i    (~e_decrement_amount),
    .cin_i  (1'b1),
    .sum_o  (e_decrement_result),
    .cout_o ()
);
...
        WAIT_N:
          begin
              if (ctrl_load == LOAD)
                  state_ns = WAIT_N;
              else if (n_reg == '0)
                  state_ns = ERROR;
              else if (e_reg == '0)
                  state_ns = CASE0;
              else if (e_reg == EXP_ONE)
                  state_ns = CASE1;
              else
                  state_ns = CASE2;
          end
```

`WAIT_N` escolhe o caso (erro, $E = 0$, $E = 1$, $E \geq 2$); o expoente é decrementado por um Kogge-Stone de 32 bits. "…" = omitido.

E02 — Equivalência lógica RSA 4 bits

Cadence Conformal LEC · golden Verilog × revised SystemVerilog

LEC — PASS

94 de 94 pontos equivalentes · 0 não equivalentes · 0 abortados

Golden	`src/vlog/rsa_core.v` (`-verilog2k`)
Revised	`src/sv/rsa_core.sv` (`-systemverilog`)
Topo	`rsa_core`
Ferramenta	Conformal 23.20-s200
Modo	`lec`, `compare_effort high`
Pontos	`add_compared_points -all`
Saída	`Compare Results: PASS` (código 0)

▶ Evidências E02 no GitHub

E02 — Equivalência lógica RSA 4 bits

Mapeamento e comparação

Estrutura elaborada (idêntica nas duas versões)

Métrica	Golden	Revised
Módulos de projeto	5	5
Entradas / saídas primárias	7 / 7	7 / 7
Primitivas (nível de bit)	866	866
Flip-flops	88	88

Comparação formal

Pontos	PO	DFF	Total
Equivalentes	7	87	**94**
Não equivalentes	0	0	0
Abortados	0	0	0

- 88 DFFs por versão: 87 mapeados e provados; **1 inalcançável** (*unreachable*) em ambas — não é ponto não equivalente.
- `report_verification`: 0 opções não padrão, 0 verificação incompleta, 0 modificação manual, 0 ambiguidade.
- Avisos de lint idênticos nos dois lados (largura de operandos, `case` sem `default`).

▶ **Evidências E02 no GitHub**

E03 — Simulação funcional do `rsa_core32b`

Compilação e execução do testbench · RSA32_opt · Xcelium 24.03

Compilação do RTL v2 e do testbench

```
TOOL:  xrun(64)  24.03-s013:
       Started on Nov 05, 2026 at 09:24:24 -03
xrun
    -top testbench
    -coverage all
    ...
file: .../src/v2/rsa32_core_kogge_stone_adder.sv
    module worklib.kogge_stone_adder:sv
        errors: 0, warnings: 0
file: .../src/v2/rsa32_core_mult.sv
    module worklib.rsa_core_mult:sv
        errors: 0, warnings: 0
file: .../src/v2/rsa32_core_mod.sv
    module worklib.rsa_core_mod:sv
        errors: 0, warnings: 0
file: .../src/v2/rsa32_core_ctrl.sv
    module worklib.rsa_core_ctrl:sv
        errors: 0, warnings: 0
file: .../src/v2/rsa32_core_top.sv
    module worklib.rsa_core:sv
        errors: 0, warnings: 0
file: .../testbenches/testbench.sv
    module worklib.testbench:sv
        errors: 0, warnings: 0
```

Execução

```
xcelium> run
------------------------------------------------------------
RSA Core - 32-bit SystemVerilog Testbench
------------------------------------------------------------
       Message    Exponent     Modulus     Result       Expected
------------------------------------------------------------
Test  1: M=00000000 E=00000000 N=00000000 -> C=ffffffff ERR=1  Expected=ffffffff ERR=1 [PASSED]
Test  2: M=00000000 E=00000000 N=00000001 -> C=00000000 ERR=0  Expected=00000000 ERR=0 [PASSED]
Test  3: M=00000000 E=00000000 N=00000002 -> C=00000001 ERR=0  Expected=00000001 ERR=0 [PASSED]
...
Test 53: M=80000000 E=00000002 N=ffffffff -> C=40000000 ERR=0  Expected=40000000 ERR=0 [PASSED]
Test 54: M=deadbeef E=00000003 N=fffffffb -> C=62fbd2ea ERR=0  Expected=62fbd2ea ERR=0 [PASSED]
Test 55: M=12345678 E=00000003 N=00000000 -> C=ffffffff ERR=1  Expected=ffffffff ERR=1 [PASSED]
------------------------------------------------------------
Tests executed : 55
Passed         : 55
Failed         : 0
------------------------------------------------------------
32-bit RSA Core validation: PASSED
------------------------------------------------------------
End of Simulation
Simulation complete via $finish(1) at time 696090 NS + 0
xcelium> exit
TOOL:  xrun(64)  24.03-s013: Exiting on Nov 05, 2026 at 09:24:29 -03  (total: 00:00:05)
```

RSA32_opt/logs/rsa_core_32_opt/xrun/rsa_core_32_opt_2026-11-05_09-24-24.log · "…" = linhas omitidas

E03 — Simulação funcional do `rsa_core32b`

`testbench.sv` · **núcleo v2 (RSA32_opt)** · **Xcelium 24.03**

55 / 55 PASSED

0 falhas · 32-bit RSA

Core validation: PASSED

55 vetores $C = M^E \bmod N$ comparados ao valor esperado: $N = 0$ (erro), operandos nulos e unitários, valores pequenos e operandos de 32 bits.

```
TOOL:  xrun(64)  24.03-s013: Started on Nov 05, 2026 at 09:26:11 -03
  -top testbench
...
Test  1: M=00000000 E=00000000 N=00000000 -> C=ffffffff ERR=1  Expected=ffffffff ERR=1 [PASSED]
Test 25: M=00000004 E=0000000a N=00000009 -> C=00000004 ERR=0  Expected=00000004 ERR=0 [PASSED]
...
Test 50: M=abcdef01 E=00000001 N=00ffffff -> C=00cdefac ERR=0  Expected=00cdefac ERR=0 [PASSED]
Test 51: M=ffffffff E=00000001 N=fffffffb -> C=00000004 ERR=0  Expected=00000004 ERR=0 [PASSED]
Test 52: M=12345678 E=00000002 N=ffffffff -> C=1f403f1c ERR=0  Expected=1f403f1c ERR=0 [PASSED]
Test 53: M=80000000 E=00000002 N=ffffffff -> C=40000000 ERR=0  Expected=40000000 ERR=0 [PASSED]
Test 54: M=deadbeef E=00000003 N=fffffffb -> C=62fbd2ea ERR=0  Expected=62fbd2ea ERR=0 [PASSED]
Test 55: M=12345678 E=00000003 N=00000000 -> C=ffffffff ERR=1  Expected=ffffffff ERR=1 [PASSED]
----------------------------------------------------------------
Tests executed : 55
Passed         : 55
Failed         : 0
----------------------------------------------------------------
32-bit RSA Core validation: PASSED
----------------------------------------------------------------
End of Simulation
Simulation complete via $finish(1) at time 696090 NS + 0
```

RSA32_opt/logs/rsa_core_32_opt/xrun/rsa_core_32_opt_2026-11-05_09-26-11.log · "..." = linhas omitidas

E04 — Ambiente de verificação UVM

Estrutura do ambiente e ligação ao DUT

- **Topo:** `top_tb` publica `apb_rsa_if` no `config_db` e chama `run_test()`; `top_th` gera `pclk` e instancia o DUT.
- **Teste:** `uvm_test_rsa_operation` (estende `uvm_test_base`) dispara a `uvm_vseq`.
- **Sequências (em ordem):** `rsa_operation` → `soft_reset` → `test_id` → `corners_exp` → `xz_values`.
- **Env:** `env_config`, `uvm_scoreboard`, `uvm_vsequencer`.
- **Agente APB:** `driver`, `monitor`, `sequencer`, `subscriber`, `agent_config`.
- **Assertions (5 módulos SVA):** `apb_protocol`, `rsa_operation`, `soft_reset`, `test_id`, `corners_exp`; resultados consolidados por `record_assertion()` e impressos em `print_assertion_table()`.
- **DUT:** `amba_3_apb_rsa_apb_top` (`src/MG`), ligado à `apb_rsa_if` pelos sinais APB e `irq`.

Diagrama da estrutura em resolução completa: `docs/UVM_structure_diagram.png` (fonte: `UVM/estrutura.md`).

E04 — Lista de testes da bancada UVM

Plano de verificação · testes 1.0 a 3.0

Id	Teste	Procedimento	Funcionalidade	Estado
1.0	`test_reset`	1–2) Reset inicial[†] 3) Escrever `0xFFFF_FFFF` em `0x0C` 4) Ler `0x0C` 5) Colocar PRESETn = 0 6) Aguardar 10 ciclos e colocar PRESETn = 1 na borda de descida do clock 7) Ler `0x0C`	1.0 — Reset externo (PRESETn)	FEITO
2.0	`test_soft_reset`	1–2) Reset inicial[†] 3) Carregar operandos: `0x0C` ← ⟨dado1⟩, `0x10` ← ⟨dado2⟩, `0x14` ← ⟨dado3⟩ 4) Disparar o start da operação: `0x00` ← `0x01` 5) Ler o registrador de status (`0x04`) 6) Aplicar soft reset: escrever em `0x00` até o valor ser `0x02` 7) Ler o registrador de status (`0x04`)	4.0 a 8.0 — Operação RSA	FEITO
3.0	`test_id`	1–2) Reset inicial[†] 3) Ler `0x1C` 4) Escrever `0xFFFF_FFFF` em `0x1C` 5) Ler `0x1C`	10.0 — Identificação do bloco	FEITO

[†] Reset inicial (testes 1.0 a 4.0): 1) iniciar o DUT com PRESETn = 0; 2) aguardar 10 ciclos de clock e colocar PRESETn = 1 na borda de descida do clock.

E04 — Lista de testes da bancada UVM

Plano de verificação · testes 4.0 a 6.0

Id	Teste	Procedimento	Funcionalidade	Estado
4.0	`test_rsa_operation`	1–2) Reset inicial[†] 3) Carregar operandos: `0x0C` ← ⟨dado1⟩, `0x10` ← ⟨dado2⟩, `0x14` ← ⟨dado3⟩ 4) Disparar o start da operação: `0x00` ← `0x01` 5) *Polling* no registrador de status: ler de `0x01` até o valor ser `0x02` 6) Ler o resultado (`0x18`)	4.0 a 8.0 — Operação RSA	FEITO
5.0	`test_corners_exp`	Casos aleatórios da operação RSA (válida / inválida / erro), com DIN e MOD aleatórios e EXP variando entre 4, 8, 12, 16 e 32 bits	Teste direcionado ao RSA de 32 bits	FEITO
6.0	`test_xz_values`	Escrita e leitura de registradores com valores X e/ou Z, utilizando endereços válidos	Teste direcionado ao APB	FEITO

[†] Reset inicial: 1) iniciar o DUT com PRESETn = 0; 2) aguardar 10 ciclos de clock e colocar PRESETn = 1 na borda de descida do clock.

Funcionalidades referenciadas (aba `Lista_de_Funcionalidades`)

1.0 Reset externo (PRESETn) · **4.0** Operação RSA ($C = M^e \bmod n$) · **5.0** Controle da operação · **6.0** Status da operação · **7.0** Dados de entrada · **8.0** Registrador de resultado · **10.0** Identificação do bloco

E05 — Simulação funcional do RSA_APB

Bancada UVM (MG) · RTL · Xcelium 24.03 · 100 MHz

TEST PASS
UVM_ERROR = 0
UVM_FATAL = 0

Execução	2026-11-10_10-24-56
Teste	`test_rsa_operation`
Topo	`amba_3_apb_rsa_apb_top`
Sequências	5 (`uvm_vseq`)
Transf. APB	109 072
Assertions	39 PASS · 0 FAIL
Simulado	3 608 310 ns

`APB_WAIT_HOLD` não exercitada: o DUT responde sem *wait states* (`PREADY` = 1).

```
TOOL:  xrun(64)  24.03-s013: Started on Nov 10, 2026 at 10:24:56 -03
  +UVM_TESTNAME=test_rsa_operation
...
UVM_INFO @ 318575000: reporter [RSA_RESULT_VALUE] MSG=0x212d0eb1 EXP=0x000000de MOD=0x2a22eb39
                                                  EXPECTED=0x07c5db67 ACTUAL=0x07c5db67
UVM_INFO @ 318580000: reporter [SEQ_RESULT] [rsa_operation] PASS | transacoes=10616 falhas_SVA=0
UVM_INFO @ 318790000: reporter [SEQ_RESULT] [soft_reset] PASS | transacoes=7 falhas_SVA=0
UVM_INFO @ 318890000: reporter [SEQ_RESULT] [test_id] PASS | transacoes=3 falhas_SVA=0
... [CORNERS_RESULT] [caso=1 EXP=4 bits VALID_OPERATION]    PASS | duracao=10280.000ns
... [CORNERS_SUMMARY] TOTAL=1 PASS=1 FAIL=0 (inclui TIMEOUT=0)
... [XZ_RESULT] CONCLUIDO | rotinas=8 enderecos=32768 transacoes=98381 falhas_resposta=0
...
--- UVM Report Summary ---
** Report counts by severity
UVM_INFO :43563
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
...
[TIMING] Tempo simulado global: 3608310.000 ns
[CAMPANHA] planejados=1 concluidos=1 interrompidos=0 nao_iniciados=0 caso_atual=1
...
[SVA] Assertions: 39 PASS | 0 FAIL | 1 NAO EXERCITADAS | 0 INTERROMPIDAS
[XZ][OBSERVACAO_NOMINAL] APB_KNOWN_READ_DATA: 256 ocorrencias intencionais de X/Z
[XZ][OBSERVACAO_NOMINAL] APB_KNOWN_WRITE_DATA: 65536 ocorrencias intencionais de X/Z
[SVA] Contagens finais UVM: ERROR=0 | FATAL=0
Simulation complete via $finish(1) at time 3608310 NS + 69
```

`xrun_uvm/nogui/amba_3_apb_rsa_apb_top_uvm_2026-11-10_10-24-56.log` · "..." = omitido

E05 — Simulação funcional do RSA_APB

Relatório final do scoreboard UVM · `print_assertion_table()`

```
+----------------+---------------------------+-------------+----------+----------+--------------+
| Cenario        | Assertion                 | Resultado   | Acertos  |  Falhas  | Instante(ns) |
+----------------+---------------------------+-------------+----------+----------+--------------+
| APB            | APB_SETUP_ACCESS          | PASS        |  109072  |        0 |  3608305.000 |
| APB            | APB_ZERO_WAIT             | PASS        |  109072  |        0 |  3608305.000 |
| APB            | APB_WAIT_HOLD             | NAO EXERC.  |        0 |        0 |           -- |
| APB            | APB_KNOWN_READ_DATA       | PASS        |    10964 |        0 |  3201635.000 |
...
| rsa_operation  | RSA_OPERATION_ORDER       | PASS        |        1 |        0 |   318575.000 |
| rsa_operation  | RSA_OPERATION_RESPONSE    | PASS        |    10616 |        0 |   318575.000 |
| rsa_operation  | RSA_RESULT_BEFORE_DONE    | PASS        |        1 |        0 |   318545.000 |
| rsa_operation  | RSA_RESULT_VALUE          | PASS        |        1 |        0 |   318575.000 |
| rsa_operation  | RSA_STATUS_VALUE          | PASS        |    10611 |        0 |   318545.000 |
...
| corners4_ OK   | CORNERS_EXP_WIDTH         | PASS        |        1 |        0 |   319015.000 |
| corners4_ OK   | CORNERS_RESPONSE          | PASS        |       64 |        0 |   329195.000 |
| corners4_ OK   | CORNERS_READBACK          | PASS        |        3 |        0 |   319125.000 |
| corners4_ OK   | CORNERS_READ_DELAY        | PASS        |        3 |        0 |   319105.000 |
| corners4_ OK   | CORNERS_W1C               | PASS        |        1 |        0 |   329195.000 |
| corners4_ OK   | CORNERS_STATUS            | PASS        |       53 |        0 |   329095.000 |
| corners4_ OK   | CORNERS_COMPLETION        | PASS        |        1 |        0 |   329095.000 |
| corners4_ OK   | CORNERS_RESULT_VALUE      | PASS        |        1 |        0 |   329125.000 |
+----------------+---------------------------+-------------+----------+----------+--------------+
[SVA] Assertions: 39 PASS | 0 FAIL | 1 NAO EXERCITADAS | 0 INTERROMPIDAS
[SVA] Contagens finais UVM: ERROR=0 | FATAL=0
```

"…" = linhas omitidas: 15 do cenário APB, `soft_reset` e `test_id` (todas PASS, 0 falhas; totais ao lado)

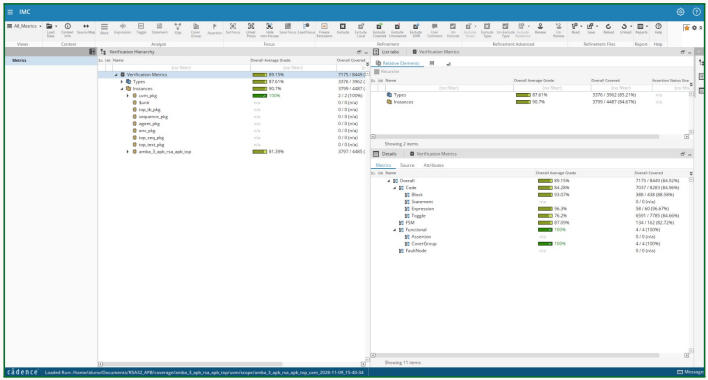
Resumo por cenário

Cenário	PASS / total
APB (protocolo)	18 / 19
`rsa_operation`	5 / 5
`soft_reset`	3 / 3
`test_id`	5 / 5
`corners_exp`	8 / 8
Total	39 / 40

0 falhas em todos os cenários; a única assertion sem PASS é `APB_WAIT_HOLD`, não exercitada. Os X/Z injetados por `xz_values` são contados à parte, como observação nominal.

E06 — Cobertura funcional do RSA_APB

Relatório de cobertura da regressão UVM · Cadence IMC



Execução `amba_3_apb_rsa_apb_top_uvm_2026-11-09_15-40-34`

Métricas (*Overall*)

Métrica	Nota	Cobertos
Geral	**89,15 %**	7 175/8 449
Código	84,28 %	7 037/8 283
Bloco	93,07 %	388/438
Expressão	96,30 %	58/60
Toggle	76,20 %	6 591/7 785
FSM	87,09 %	134/162
Funcional	**100 %**	4/4

- DUT (`amba_3_apb_rsa_apb_top`): 81,39 % (3 797/4 485).
- Cobertura funcional: 4 de 4 *covergroups* atingidos.
- Toggle é a métrica mais baixa (76,2 %).

E07 — Gate-level netlist TSMC 28 nm

Recorte do netlist da última síntese · Genus 22.18 · `write_hdl`

```
// Generated by Cadence Genus(TM) Synthesis Solution 22.18-s061_1
// Generated on: Nov  9 2026 15:16:53 -03 (Nov  9 2026 18:16:53 UTC)

module amba_3_apb_rsa_apb_top(pclk_i, presetn_i, psel_i, penable_i,
     pwrite_i, paddr_i, pwdata_i, prdata_o, pready_o, pslverr_o, irq_o);
  input pclk_i, presetn_i, psel_i, penable_i, pwrite_i;
  input [11:0] paddr_i;
  input [31:0] pwdata_i;
  output [31:0] prdata_o;
  output pready_o, pslverr_o, irq_o;
  ...
  assign pready_o = 1'b1;
  ...
  XOR2UD1BWP30P140 g81243__3680(.A1 (n_1795), .A2 (n_1794), .Z
       (n_1796));
  ...
  EDFCNQD1BWP30P140 \regbank_inst_exp_reg_reg[20] (.CDN (presetn_i),
       .CP (pclk_i), .D (pwdata_i[20]), .E (n_1431), .Q (op_exp_w[20]));
  ...
  OR2D1BWP30P140 g84438(.A1 (n_339), .A2 (n_407), .Z (n_1962));
endmodule
```

`synth_amba_3_apb_rsa_apb_top_2026-11-09_15-16-04_620008766_tsmc28.v` · 8 797 linhas

Síntese `2026-11-09_15-16-04`

Biblioteca	`tcbn28hpcplusbwp30p140`
Corner	tt1v25c
Módulos	1 (hierarquia achatada)
Células	2 562
Área	2 808,16 μm²

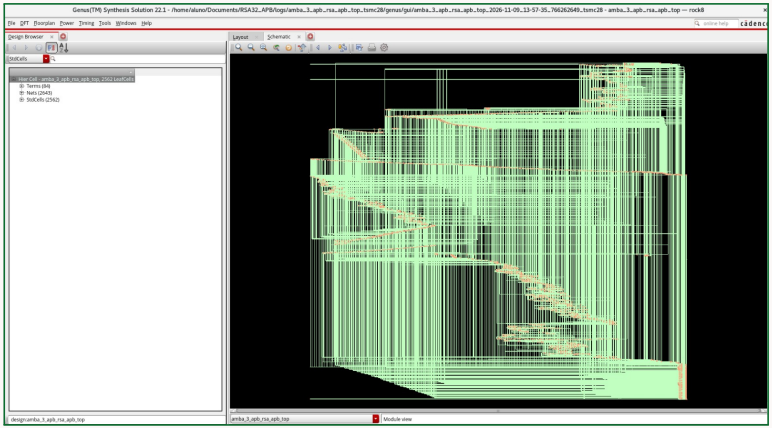
Composição (`gates_tsmc28.rpt`)

Tipo	Instâncias	Área
Sequencial	706	62,2 %
Inversor	176	1,6 %
Lógica	1 680	36,2 %
Total	2 562	100 %

Todas as células instanciadas pertencem à biblioteca TSMC (sufixo `BWP30P140`).

E07 — Gate-level netlist TSMC 28 nm

Esquemático do netlist sintetizado · Cadence Genus 22.1



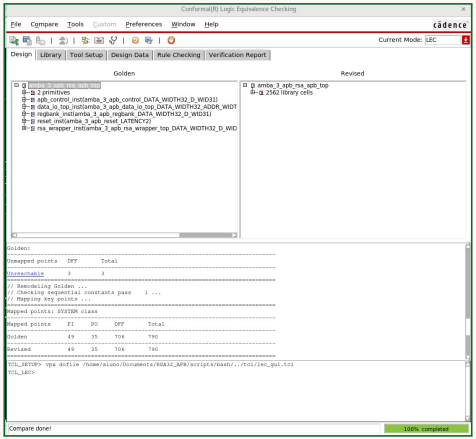
Netlist `amba_3_apb_rsa_apb_top`

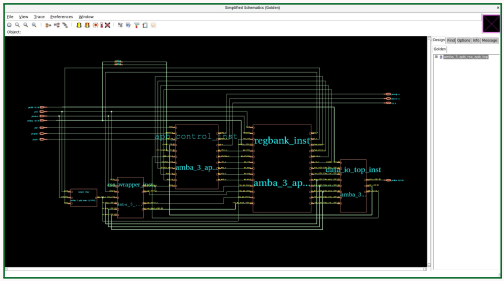
Células padrão	2 562
Nets	2 643
Terminais	84

- Topo achatado: apenas células folha da biblioteca TSMC 28 nm.
- 706 sequenciais / 1 856 combinacionais (ver E08).

E07 — Gate-level netlist TSMC 28 nm

Equivalência RTL × netlist · Cadence Conformal LEC





Esquemático simplificado do *golden* (RTL) no Conformal.

- **Golden:** RTL (5 instâncias + 2 primitivas) · **Revised:** netlist com 2 562 células.
- Pontos mapeados idênticos nos dois lados: 49 PI · 35 PO · 706 DFF = **790**.
- 3 DFF inalcançáveis (*unreachable*) no golden, não mapeados.

SEM VIOLAÇÕES DE SETUP

WNS = +5 469,7 ps · TNS = 0 · 0 caminhos violados

Síntese	`2026-11-05_14-36-09`
Clock `pclk_i`	10 ns (100 MHz), incerteza 200 ps
I/O	`input_delay` = `output_delay` = 2 ns
Corner	tt1v25c · fio *segmented* (pré-layout)
Caminhos detalhados	100, todos **MET**
atraso externo / setup	33 / 67

▶ **Evidências E08 no GitHub**

E08 — Análise estática de temporização

Caminho crítico e interpretação

Path 1: `paddr_i[6]` → `pslverr_o`

Borda de captura	10 000 ps
− `output_delay`	2 000 ps
− incerteza	200 ps
Tempo requerido	7 800 ps
`input_delay`	2 000 ps
+ caminho de dados (19 células)	330 ps
Tempo de chegada	2 330 ps
Slack	**+5 470 ps**

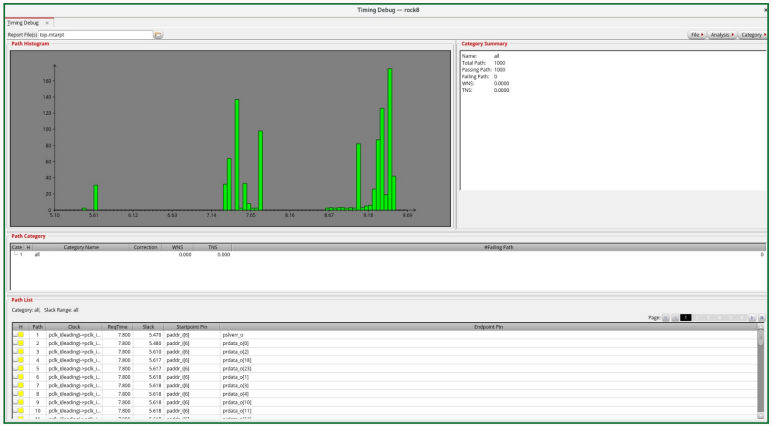
Decodificação de endereço do APB (`NR4D0`, `ND2D1`, `AOI21...`) até a saída de erro.

- Os 33 piores caminhos são de **I/O**: o orçamento de 2 ns + 2 ns das restrições externas domina.
- Pior verificação de *setup* em registrador: `paddr_i` → `regbank_inst_exp_reg[18]/E`, slack **+7 338 ps**.
- Área: 2 808,16 μm², 2 562 células (706 seq. / 1 856 comb.).

▶ **Evidências E08 no GitHub**

E08 — Análise estática de temporização

Timing Debug · histograma de slack e piores caminhos



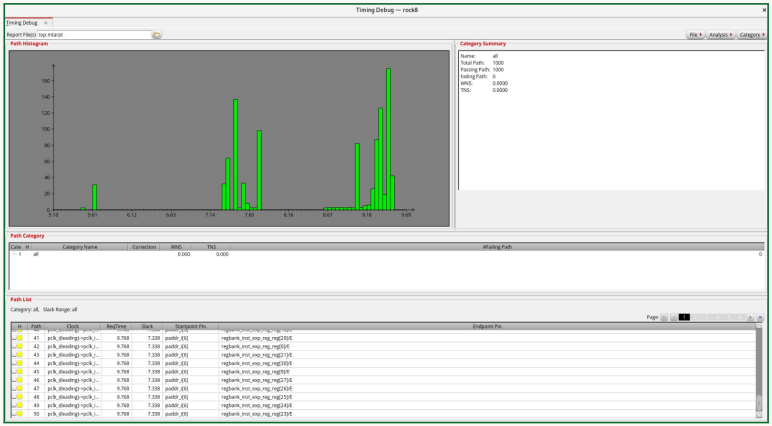
Caminhos	1 000
Aprovados	1 000
Violados	**0**
WNS / TNS	0 / 0

- Slack entre ≈5,1 e 9,7 ns.
- Pior caminho: `paddr_i[6]` → `pslverr_o`, slack **5,470 ns** (requerido 7,800 ns).
- Seguintes: `paddr_i[6]` → `prdata_o[*]`, slack ≥ 5,480 ns.

▶ **Evidências E08 no GitHub**

E08 — Análise estática de temporização

Timing Debug · caminhos internos (setup em registrador)



- Caminhos 41–50: `paddr_i[6]` → `regbank_inst_exp_reg_reg[*]/E`.
- Requerido 9,768 ns · slack **7,338 ns**.
- Habilitação de escrita do registrador `EXP`: pior setup em registrador, com folga ampla.

▶ **Evidências E08 no GitHub**

Comparativo MG × MR — síntese TSMC 28 nm

Cadence Genus 22.18 · TSMC 28 nm · tt1v25c · 100 MHz

Métrica	MG	MR
Área total	2 808 μm²	2 694 μm² ($-4{,}1\,\%$)
Células combinacionais	1 680	1 324 ($-21\,\%$)
Potência estimada	529 μW	500 μW ($-5{,}5\,\%$)
Atraso do núcleo (reg $\rightarrow$ reg)	**1 114 ps**	1 547 ps
Frequência indicativa pelo núcleo	$\approx$874 MHz	$\approx$642 MHz
Pior slack	+5 470 ps	+3 153 ps
Endpoints com slack $<$ 5 ns	0	33

Sínteses `2026-11-10_09-21-01` (MG) e `2026-11-10_09-31-35` (MR), ambas sem violações. Potência estimada pelo Genus sem vetores de atividade. Frequência indicativa = 1 / (atraso + setup) do caminho reg $\rightarrow$ reg do núcleo, sem incerteza. O pior slack depende do SDC de cada projeto: MR com I/O de 3 ns + 3 ns e incerteza de 0,5 ns; MG com 2 ns + 2 ns e 0,2 ns.

E09 — Simulação pós-síntese com SDF

Gate-level TSMC 28 nm · Xcelium 24.03 · 100 MHz

TEST PASS — 398 / 398

0 *Timing violation* · 0 mensagens *E/*F · exit 0

Simulação	`2026-11-05_14-37-30`
Netlist / SDF	síntese `2026-11-05_14-36-09` (E08)
Biblioteca	`tcbn28hpcplusbwp30p140`
Corner	TT · 1,0 V · 25 °C · SDF `MAXIMUM`
Defines	`POST_SYNTHESIS`, `NTC`, `RECREM`
Tempo simulado	134 980 ns

▶ Evidências E09 no GitHub

E09 — Simulação pós-síntese com SDF

Anotação SDF e correlação com o RTL

Estatísticas de anotação

Categoria	Modelo	Anotados	%
Atrasos de caminho	14 871	14 871	**100,00**
`$setuphold`	7 314	7 312	99,97
`$recrem`	140	140	100,00
`$width`	10 372	0	0,00
Checks (total)	17 826	7 452	41,80

100 % do que o SDF fornece foi anotado. O SDF não contém entradas `WIDTH`; os 2 `$setuphold` restantes são do FF de sincronização do reset com `D = 1'b1`.

RTL × gate-level (E09)

Com o testbench dirigido `testbench.sv` (execução RTL `2026-11-05_15-10-39`), as 398 linhas dos dois scoreboards são idênticas: valores esperados, lidos, status e instantes de verificação.

▶ Evidências E09 no GitHub

E10 — Scripts de automação

Árvore de `scripts/` (1/4) · `MG/bash`

```
scripts/
├── clean.sh
├── MG
│   ├── bash
│   │   ├── 0.innovus_init.sh
│   │   ├── 1.innovus_check.sh
│   │   ├── 2.innovus_floorplan.sh
│   │   ├── 3.innovus_power.sh
│   │   ├── 4.innovus_place.sh
│   │   ├── 5.innovus_cts.sh
│   │   ├── 6.innovus_route.sh
│   │   ├── 7.innovus_postroute.sh
│   │   ├── 8.innovus_verify.sh
│   │   ├── 9.innovus_export.sh
│   │   ├── genus_gui.sh
│   │   ├── genus_no_gui.sh
│   │   ├── genus_nogui.sh
│   │   ├── imc_common.sh
│   │   ├── imc_gui.sh
│   │   ├── imc.key
│   │   ├── imc.log
│   │   ├── imc.sh
│   │   ├── innovus_common.sh
│   │   ├── innovus_full_flow.sh
│   │   ├── innovus_gui.sh
│   │   ├── innovus_no_gui.sh
│   │   ├── iverilog.sh
│   │   ├── lec_gui.sh
│   │   ├── lec.sh
│   │   ├── lint.sh
│   │   ├── mdv.log
│   │   ├── pdk_common.sh
│   │   ├── report_common.sh
│   │   ├── run
│   │   ├── setup_uvm.sh
│   │   ├── sky130_check.sh
│   │   ├── source
│   │   ├── uvm_campaign.sh
│   │   ├── verilator.sh
│   │   ├── xezim_rsa_core_fault.sh
│   │   ├── xezim.sh
│   │   ├── xezim_uvm.sh
│   │   ├── xrun_ps.sh
│   │   ├── xrun_ps_tsmc28.sh
│   │   ├── xrun.sh
│   │   ├── xrun_uvm_gui.sh
│   │   ├── xrun_uvm_nogui.sh
│   │   └── xrun_wgui.sh
```

Etapas cobertas pelos *wrappers*

Síntese	`genus_*.sh`
LEC	`lec.sh`, `lec_gui.sh`
Lint	`lint.sh`, `verilator.sh`
Sim. RTL	`xrun.sh`, `xrun_wgui.sh`, `iverilog.sh`
UVM	`xrun_uvm_*.sh`, `setup_uvm.sh`, `uvm_campaign.sh`
Pós-síntese	`xrun_ps.sh`, `xrun_ps_tsmc28.sh`
Cobertura	`imc*.sh`
Ondas	`xezim*.sh`
Físico	`0–9.innovus_*.sh`, `innovus_full_flow.sh`
Comuns	`pdk_`, `report_`, `innovus_`, `imc_common.sh`

Cada *wrapper* chama o script TCL correspondente em `scripts/MG/tcl` ou `scripts/MR/tcl` (ex.: Genus executa `source scripts/MG/bash/../tcl/run_genus_tsmc28.tcl`).

E10 — Scripts de automação

Árvore de `scripts/` (2/4) · `MG/tcl`

```
└── tcl
    ├── 1.innovus_check.tcl
    ├── 2.innovus_floorplan.tcl
    ├── 3.innovus_power.tcl
    ├── 4.innovus_place.tcl
    ├── 5.innovus_cts.tcl
    ├── 6.innovus_route.tcl
    ├── 7.innovus_postroute.tcl
    ├── 8.innovus_verify.tcl
    ├── 9.innovus_export.tcl
    ├── imc.tcl
    ├── innovus_common.tcl
    ├── innovus_gui.tcl
    ├── innovus_init.tcl
    ├── innovus_mmmc.tcl
    ├── lec_common.tcl
    ├── lec_gui.tcl
    ├── lec.tcl
    ├── lint.tcl
    ├── open_genus_gui.tcl
    ├── pdk_common.tcl
    ├── pdks
    │   ├── free45.tcl
    │   ├── gpdk45.tcl
    │   ├── ics55.tcl
    │   ├── sky130.tcl
    │   └── tsmc28.tcl
    ├── report_common.tcl
    ├── run_genus_free45.tcl
    ├── run_genus_gpdk45.tcl
    ├── run_genus_ics55.tcl
    ├── run_genus_sky130.tcl
    ├── run_genus.tcl
    ├── run_genus_tsmc28.tcl
    ├── run_innovus_free45.tcl
    ├── run_innovus_gpdk45.tcl
    ├── run_innovus_ics55.tcl
    ├── run_innovus_sky130.tcl
    ├── xezim_uvm_wave_view.tcl
    ├── xrun_ps_tsmc28.tcl
    ├── xrun_uvm_gui.tcl
    ├── xrun_uvm_nogui.tcl
    └── xrun_wgui.tcl
```

- `pdks/`: um arquivo por tecnologia (TSMC 28 nm, GPDK45, FreePDK45, ICS55, SKY130).
- `run_genus_<pdk>.tcl` e `run_innovus_<pdk>.tcl`: entradas por tecnologia sobre os fluxos comuns.
- `lec_common.tcl`, `report_common.tcl`, `pdk_common.tcl`: código compartilhado entre ferramentas.

E10 — Scripts de automação

Árvore de scripts/ (3/4) · MR/bash

```
MR
└── bash
    ├── 0.innovus_init.sh
    ├── 1.innovus_check.sh
    ├── 2.innovus_floorplan.sh
    ├── 3.innovus_power.sh
    ├── 4.innovus_place.sh
    ├── 5.innovus_cts.sh
    ├── 6.innovus_route.sh
    ├── 7.innovus_postroute.sh
    ├── 8.innovus_verify.sh
    ├── 9.innovus_export.sh
    ├── genus_gui.sh
    ├── genus_no_gui.sh
    ├── genus_nogui.sh
    ├── imc_common.sh
    ├── imc_gui.sh
    ├── imc.sh
    ├── innovus_common.sh
    ├── innovus_full_flow.sh
    ├── innovus_gui.sh
    ├── innovus_no_gui.sh
    ├── iverilog.sh
    ├── lec_gui.sh
    ├── lec.sh
    ├── lint.sh
    ├── pdk_common.sh
    ├── report_common.sh
    ├── setup_uvm.sh
    ├── sky130_check.sh
    ├── uvm_campaign.sh
    ├── verilator.sh
    ├── xezim_rsa_core_fault.sh
    ├── xezim.sh
    ├── xezim_uvm.sh
    ├── xrun_ps.sh
    ├── xrun_ps_tsmc28.sh
    ├── xrun.sh
    ├── xrun_uvm_gui.sh
    ├── xrun_uvm_nogui.sh
    └── xrun_wgui.sh
```

- `MR/` replica os fluxos de `MG/` com o RTL de `src/MR/RSA_APB` e o SDC `constraints/MR/rsa_apb.sdc`.
- Saídas separadas em `logs/MR`, `reports/MR`, `waveforms/MR`, `coverage/MR` e `ps_files/MR`.
- `xrun.sh` e `xrun_wgui.sh` usam o TB do MR (`rsa_apb_tb.sv`); UVM e `xrun_ps_tsmc28.sh` usam as bancadas comuns.

E10 — Scripts de automação

Árvore de `scripts/` (4/4) · MR/tcl, RSA32, RSA4 e comuns

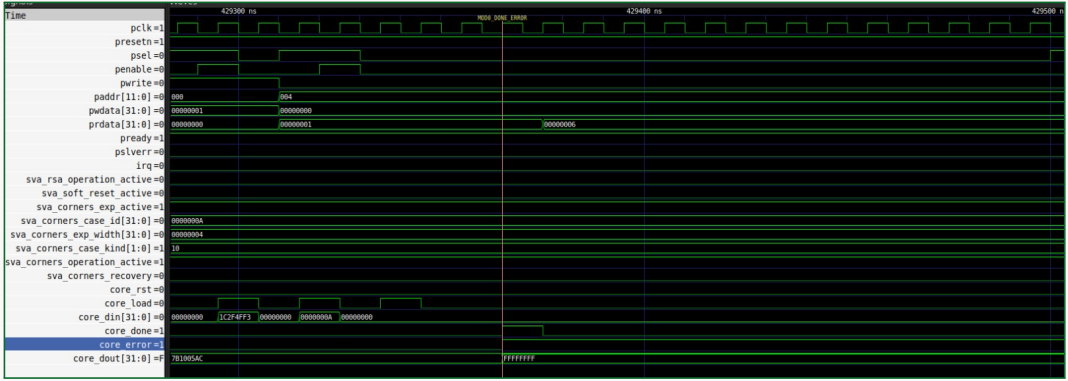
```
└── tcl
    ├── 1.innovus_check.tcl
    ├── 2.innovus_floorplan.tcl
    ├── 3.innovus_power.tcl
    ├── 4.innovus_place.tcl
    ├── 5.innovus_cts.tcl
    ├── 6.innovus_route.tcl
    ├── 7.innovus_postroute.tcl
    ├── 8.innovus_verify.tcl
    ├── 9.innovus_export.tcl
    ├── imc.tcl
    ├── innovus_common.tcl
    ├── innovus_gui.tcl
    ├── innovus_init.tcl
    ├── innovus_mmmc.tcl
    ├── lec_common.tcl
    ├── lec_gui.tcl
    ├── lec.tcl
    ├── lint.tcl
    ├── open_genus_gui.tcl
    ├── pdk_common.tcl
    ├── pdks
    │   ├── free45.tcl
    │   ├── gpdk45.tcl
    │   ├── ics55.tcl
    │   ├── sky130.tcl
    │   └── tsmc28.tcl
    ├── report_common.tcl
    ├── run_genus_free45.tcl
    ├── run_genus_gpdk45.tcl
    ├── run_genus_ics55.tcl
    ├── run_genus_sky130.tcl
    ├── run_genus.tcl
    ├── run_genus_tsmc28.tcl
    ├── run_innovus_free45.tcl
    ├── run_innovus_gpdk45.tcl
    ├── run_innovus_ics55.tcl
    ├── run_innovus_sky130.tcl
    ├── xezim_uvm_wave_view.tcl
    ├── xrun_ps_tsmc28.tcl
    ├── xrun.tcl
    ├── xrun_uvm_gui.tcl
    ├── xrun_uvm_nogui.tcl
    └── xrun_wgui.tcl
├── pages
│   ├── build.py
│   ├── coverage_theme.py
│   └── documents.py
│   └── requirements.txt
├── README.md
├── README_SKY130.md
├── RSA32
│   ├── bash
│   │   ├── xrun_ps.sh
│   │   ├── xrun.sh
│   │   └── xrun_wgui.sh
│   └── tcl
│       └── xrun_wgui.tcl
├── RSA4
│   ├── bash
│   │   └── run_rsa_lec.sh
│   └── tcl
│       └── rsa_lec.tcl
└── validation
    ├── report_xezim_coverage.py
    ├── test_uvm_campaign_timing.py
    ├── uvm_campaign_timing.py
    └── xezim_coverage.html
```

`RSA32/`: simulação Xcelium do `rsa_core` de 32 bits (RTL e TB em `RSA32_opt`).

`RSA4/`: LEC RTL × RTL do RSA de 4 bits (`.v` × `.sv`).

Falhas encontradas no RSA32

Operação RSA com módulo igual a zero



Caso 10 da `corners_exp`: MSG = `0x1C2F4FF3`, EXP = `0x0000000A`, MOD = `0x00000000`. Em 429 365 ns, `core_done` e `core_error` sobem juntos e `core_dout` passa de `0x7B1005AC` para `0xFFFFFFFF`.

O problema aparece quando tentamos executar uma operação RSA com o módulo igual a zero. Nessa situação, o esperado é que o circuito sinalize o erro e mantenha o resultado da operação anterior. O que vimos, porém, foi a ativação dos sinais de conclusão e erro ao mesmo tempo. Com isso, o registrador de status indica as duas condições, e o resultado anterior acaba substituído por `0xFFFFFFFF`.

Também percebemos que o sinal de erro continua ativo até aplicarmos um reset. No teste com a versão v2, tentamos limpar os eventos pelo registrador de status: o sinal de conclusão foi limpo, mas o erro permaneceu, mantendo a interrupção ativa.

Também testamos a operação com **expoente igual a zero**, aplicando **DIN =** `0x181D60BF` e módulo `0xC2065569`. Nesse caso, o resultado esperado é `0x00000001`. Esse é um cenário diferente da falha descrita, que ocorre quando o **módulo** é zero.

A falha apareceu nas duas versões do RSA e foi reproduzida no testbench sem UVM ou SVAs. Isso mostra que as assertions estão apontando um comportamento que realmente ocorre na simulação. O próximo passo é localizar, no RTL, por que uma operação inválida sinaliza conclusão e por que o erro permanece ativo.